

UNIVERSIDAD AMERICANA
Facultad de Ingeniería y Arquitectura



Programación Orientada a Objetos II

Pruebas Unitarias Básicas: Calculadora

Estudiante:

- Sara Alejandra Zambrana Taylor

Docente:

- José Durán García

Managua, Nicaragua

(09/06/2026)

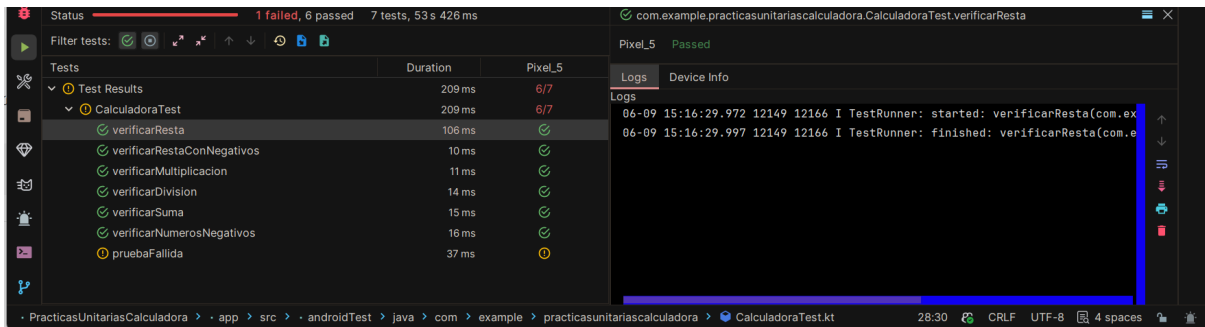
I. Evidencias

I. I App funcionando

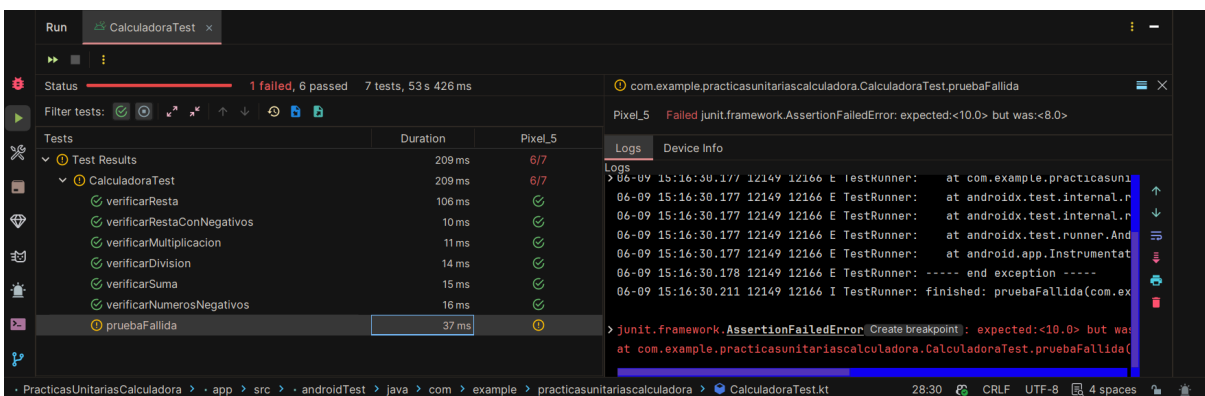


I. II Pruebas Unitarias

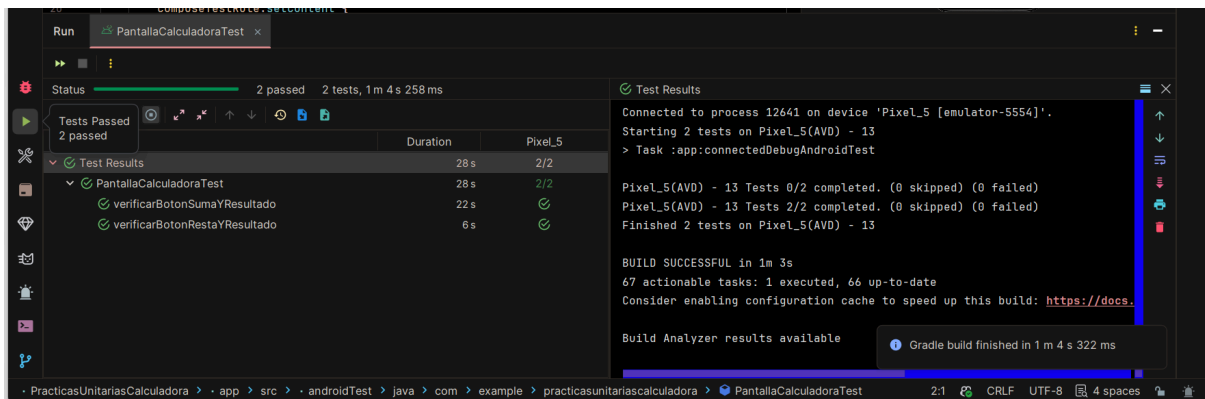
I. II. I Pruebas de operaciones



I. II. II Prueba Fallida Intencional



I. III Prueba de interfaz



I. IV Código fuente del proyecto

models/Calculadora

```
package com.example.practicasunitariascalculadora.models
```

```
class Calculadora {  
  
    fun sumar(a: Double, b: Double): Double {  
        return a + b  
    }  
  
    fun restar(a: Double, b: Double): Double {
```

```

        return a - b
    }

    fun multiplicar(a: Double, b: Double): Double {
        return a * b
    }

    fun dividir(a: Double, b: Double): Double {

        require(b != 0.0) {
            "No se puede dividir entre cero"
        }

        return a / b
    }
}

```

screens/Calculadora

```

package com.example.practicasunitariascalculadora.screens

import androidx.compose.foundation.background
import androidx.compose.foundation.border
import androidx.compose.foundation.layout.*
import androidx.compose.foundation.shape.RoundedCornerShape
import androidx.compose.material3.Button
import androidx.compose.material3.ButtonDefaults
import androidx.compose.material3.Text
import androidx.compose.runtime.*
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.draw.clip
import androidx.compose.ui.graphics.Color
import androidx.compose.ui.platform.testTag
import androidx.compose.ui.text.font.FontWeight
import androidx.compose.ui.unit.dp
import androidx.compose.ui.unit.sp
import com.example.practicasunitariascalculadora.models.Calculadora

@Composable
fun PantallaCalculadora() {

    val calculadora = Calculadora()

    var resultado by remember {
        mutableStateOf("")
    }

    var operacion by remember {
        mutableStateOf("")
    }

    val fondoPantalla = Color(0xFFE8E3D7)
    val colorBoton = Color(0xFFFF3F0E6)

```

```

val colorSombra = Color(0xFF2E2E2E)
val colorPanel = Color(0xFFBDE6D2)

Box(
    modifier = Modifier
        .fillMaxSize()
        .background(fondoPantalla),
    contentAlignment = Alignment.Center
) {

    // sombra
    Box(
        modifier = Modifier
            .offset(x = 8.dp, y = 8.dp)
            .width(320.dp)
            .height(450.dp)
            .background(
                colorSombra,
                RoundedCornerShape(24.dp)
            )
    )

    // tarjeta principal
    Column(
        modifier = Modifier
            .width(320.dp)
            .height(450.dp)
            .background(
                Color(0xFFFF6F2E9),
                RoundedCornerShape(24.dp)
            )
            .border(
                2.dp,
                Color.Black,
                RoundedCornerShape(24.dp)
            )
    ) {

        // area del resultado
        Column(
            modifier = Modifier
                .fillMaxWidth()
                .weight(1f)
                .padding(24.dp),
            horizontalAlignment = Alignment.End,
            verticalArrangement = Arrangement.Center
        ) {

            Text(
                text = if (resultado.isEmpty()) "--" else resultado,
                modifier = Modifier.testTag("resultado"),
                fontSize = 32.sp,
                fontWeight = FontWeight.Bold
            )
        }
    }
}

```

```

    )

    Spacer(
        modifier = Modifier.height(8.dp)
    )

    Text(
        text = operacion,
        fontSize = 16.sp,
        color = Color.DarkGray
    )
}

// area de botones
Column(
    modifier = Modifier
        .fillMaxWidth()
        .weight(2f)
        .clip(
            RoundedCornerShape(
                topStart = 24.dp,
                topEnd = 24.dp,
                bottomStart = 24.dp,
                bottomEnd = 24.dp
            )
        )
    )
    .background(colorPanel)
    .padding(16.dp),
    verticalArrangement = Arrangement.spacedBy(12.dp)
) {

    Row(
        horizontalArrangement = Arrangement.spacedBy(12.dp)
    ) {

        BotonOperacion(
            texto = "Sumar",
            colorBoton = colorBoton
        ) {

            val res =
                calculadora.sumar(5.0, 3.0)

            resultado =
                formatearNumero(res)

            operacion = "5 + 3"
        }

        BotonOperacion(
            texto = "Restar",
            colorBoton = colorBoton
        ) {

```

```

        val res =
            calculadora.restar(5.0, 3.0)

        resultado =
            formatearNumero(res)

        operacion = "5 - 3"
    }
}

Row(
    horizontalArrangement = Arrangement.spacedBy(12.dp)
) {

    BotonOperacion(
        texto = "Multiplicar",
        colorBoton = colorBoton
    ) {

        val res =
            calculadora.multiplicar(5.0, 3.0)

        resultado =
            formatearNumero(res)

        operacion = "5 x 3"
    }

    BotonOperacion(
        texto = "Dividir",
        colorBoton = colorBoton
    ) {

        val res =
            calculadora.dividir(6.0, 3.0)

        resultado =
            formatearNumero(res)

        operacion = "6 / 3"
    }
}
}

}

}

}

// elimina los .0 cuando el numero es entero
private fun formatearNumero(
    numero: Double
): String {

```

```

        return if (numero % 1.0 == 0.0) {
            numero.toInt().toString()
        } else {
            numero.toString()
        }
    }
}

@Composable
private fun RowScope.BotonOperacion(
    texto: String,
    colorBoton: Color,
    onClick: () -> Unit
) {
    Button(
        onClick = onClick,
        modifier = Modifier
            .weight(1f)
            .height(70.dp),
        shape = RoundedCornerShape(16.dp),
        colors = ButtonDefaults.buttonColors(
            containerColor = colorBoton,
            contentColor = Color.Black
        )
    ) {
        Text(
            text = texto,
            fontWeight = FontWeight.Bold
        )
    }
}

```

MainActivity

```

package com.example.practicasunitariascalculadora

import android.os.Bundle
import androidx.activity.ComponentActivity
import androidx.activity.compose.setContent
import androidx.activity.enableEdgeToEdge
import androidx.compose.foundation.layout.fillMaxSize
import androidx.compose.foundation.layout.padding
import androidx.compose.material3.Scaffold
import androidx.compose.material3.Text
import androidx.compose.runtime.Composable
import androidx.compose.ui.Modifier
import androidx.compose.ui.tooling.preview.Preview
import com.example.practicasunitariascalculadora.screens.PantallaCalculadora
import
com.example.practicasunitariascalculadora.ui.theme.PracticasUnitariasCalculad
oraTheme

class MainActivity : ComponentActivity() {

```



```

override fun onCreate(savedInstanceState: Bundle?) {
    super.onCreate(savedInstanceState)
    enableEdgeToEdge()
    setContent {
        PantallaCalculadora()
    }
}
}

```

CalculadoraTest

```

package com.example.practicasunitariascalculadora

import com.example.practicasunitariascalculadora.models.Calculadora
import junit.framework.TestCase.assertEquals
import org.junit.Assert
import org.junit.Test

class CalculadoraTest {

    private val calculadora = Calculadora()

    @Test
    fun verificarSuma() {
        //delta:margen de error permitido
        assertEquals(8.0, calculadora.sumar(5.0, 3.0), 0.001) //"La prueba
        //pasa si el resultado está entre 7.999 y 8.001"
    }

    @Test
    fun verificarResta() {
        assertEquals(2.0, calculadora.restar(5.0, 3.0), 0.001)
    }

    @Test
    fun verificarMultiplicacion() {
        assertEquals(15.0, calculadora.multiplicar(5.0, 3.0), 0.001)
    }

    @Test
    fun verificarDivision() {
        assertEquals(1.666, calculadora.dividir(5.0, 3.0), 0.001)
    }

    @Test
    fun verificarNumerosNegativos() {
        assertEquals(-8.00, calculadora.sumar(-5.00, -3.00))
    }

    @Test
    fun verificarRestaConNegativos() {
        assertEquals(-2.00, calculadora.restar(-5.00, -3.00))
    }
}

```

```

    }

    // Prueba para fallar intencionalmente
    @Test
    fun pruebaFallida() {
        assertEquals(10.00, calculadora.sumar(5.00, 3.00))
    }
}

```

PantallaCalculadoraTest

```

package com.example.practicasunitariascalculadora

import androidx.compose.ui.test.assertTextEquals
import androidx.compose.ui.test.junit4.createComposeRule
import androidx.compose.ui.test.onNodeWithTag
import androidx.compose.ui.test.onNodeWithText
import androidx.compose.ui.test.performClick
import com.example.practicasunitariascalculadora.screens.PantallaCalculadora
import org.junit.Rule
import org.junit.Test

class PantallaCalculadoraTest {

    @get:Rule
    val composeTestRule = createComposeRule()

    @Test
    fun verificarBotonSumaYResultado() {

        composeTestRule.setContent {
            PantallaCalculadora()
        }

        composeTestRule
            .onNodeWithText("Sumar")
            .performClick()

        composeTestRule
            .onNodeWithTag("resultado")
            .assertTextEquals("8")
    }

    @Test
    fun verificarBotonRestaYResultado() {

        composeTestRule.setContent {
            PantallaCalculadora()
        }

        composeTestRule
            .onNodeWithText("Restar")
            .performClick()
    }
}

```

```
composeTestRule
    .onNodeWithTag("resultado")
    .assertTextEquals("2")
}
```

II. Documentación

¿Qué válida la prueba unitaria?

La prueba unitaria válida que los métodos de la clase 'Calculadora' (suma, resta, multiplicación, división). Devuelvan los resultados esperados para determinados valores de entrada. También, se ejecutan las pruebas complementarias sugeridas, como la verificación con números fallidos y una prueba fallida intencional.

¿Qué válida la prueba de interfaz?

Esta valida que los componentes visuales funcionen correctamente. En este caso, se verifica que al presionar los diferentes botones, se ejecuten las acciones correspondientes y se muestre el resultado esperado en pantalla.

Diferencias entre ambos tipos de pruebas

Las pruebas unitarias evalúan el funcionamiento interno de la lógica del programa de forma aislada y sin necesidad de ejecutar la interfaz. Por otro lado, las pruebas de interfaz verifican la interacción del usuario con los elementos visuales y comprueban que la aplicación responda correctamente a dichas acciones.

III. Preguntas de reflexión

1. ¿Cuál es la finalidad de las pruebas unitarias en el desarrollo de aplicaciones móviles?

La finalidad de estas, es verificar que cada componente o función de la app funcione correctamente de manera independiente, permitiendo de esta manera detectar errores en la lógica del programa desde etapas tempranas del desarrollo.

2. ¿Qué ventajas ofrece Jetpack Compose para realizar pruebas de interfaz?

Jetpack Compose facilita las pruebas de interfaz mediante herramientas que permiten identificar, interactuar y validar componentes visuales de forma sencilla, haciendo que las pruebas sean más rápidas y fáciles de manejar.

3. ¿Qué problemas pueden detectarse mediante pruebas automatizadas?

Las pruebas automatizadas pueden detectar errores de cálculo, fallos en la lógica de negocio, comportamientos inesperados de la interfaz y problemas que puedan surgir después de realizar cambios en el código.

4. ¿Por qué es importante ejecutar pruebas antes de publicar una aplicación?

Porque ayudan a garantizar que la aplicación funcione correctamente, reducen la probabilidad de errores en producción y mejoran la calidad y confiabilidad del software para los usuarios.

5. ¿Qué diferencia existe entre probar la lógica de negocio y probar la interfaz gráfica?

Probar la lógica de negocio consiste en verificar que los cálculos y procesos internos generen resultados correctos, mientras que probar la interfaz gráfica consiste en comprobar que los elementos visuales respondan adecuadamente a las acciones del usuario y muestren la información esperada.